สาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยเทคโนโลยีสุรนารี

วิชา ENG23 2011 ระบบฐานข้อมูล (Database Systems)

ปฏิบัติการที่ 7 MySQL Stored Procedures and MySQL Stored Function

<u>จุดประสงค์การปฏิบัติการ</u>

- นักศึกษาสามารถเขียน Stored Procedures ได้
- นักศึกษาสามารถเขียน Stored Function ได้

<u>ไฟล์ที่จำเป็น</u> : https://bit.ly/3cp72IA

1. MySQL Stored Procedures

การเขียน Stored Procedure มีความสำคัญมากในการพัฒนาระบบขนาดใหญ่ ที่มีการจัดเก็บข้อมูลและประมวลผลจำนวนมาก ๆ เพราะการสอบถามข้อมูล (Query) ในรูปแบบปกติที่มีการติดต่อกับข้อมูลหลาย ๆ ครั้งนั้น จำเป็นจะต้อง Select แล้วใช้โปรแกรมอ่านค่า เมื่ออ่านได้ค่าแล้วค่อยส่งไปประมวลผลที่ Database ซ้ำ ๆ ซึ่งจะเป็นการทำงานซ้ำซ้อน ส่งผลให้ประสิทธิภาพของโปรแกรมจะทำงานช้ามาก ทางเลือกในการแก้ปัญหานี้ คือ ใช้การทำงานซ้ำซ้อนทั้งหมดนี้ที่ Database แทน โดยเพียงส่งค่า Parameters ที่จำเป็นต้องใช้ จากนั้นบน Database ก็จะนำค่า Parameters ที่ส่งไปนั้น ทำงานตามคำสั่งต่าง ๆ บน Stored Procedure ที่เขียนขึ้น เมื่อได้ค่าที่ต้องการค่อยส่งค่า Result กลับมายังโปรแกรม วิธีนี้จะเป็นการเพิ่มประสิทธิภาพการทำงานของโปรแกรมให้ทำงานเร็วขึ้น และลด Traffic ระหว่าง Database กับ Application ได้สูงมาก

- คำสั่งสร้าง stored procedure

```
CREATE PROCEDURE stored_procedure_name
([IN | OUT | INOUT] parameter_name datatype[(length)])
BEGIN
    statements;
END;
```

- คำสั่งลบ stored procedure

```
DROP PROCEDURE [IF EXISTS] stored_procedure_name;
```

- คำสั่งดู stored procedure ในระบบ

```
SHOW PROCEDURE STATUS [LIKE 'pattern' | WHERE search_condition]
```

ตัวอย่าง

```
CREATE PROCEDURE `getAllCustomer`()
BEGIN
    SELECT * FROM customer;
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`getAllCustomer`();
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

customer_name	customer_stre...	customer_city
Dang	Univ	Korat
Som	Mitr-Pab	Korat

2. MySQL Declaring variables

- คำสั่งสำหรับการประกาศตัวแปร

```
DECLARE variable_name datatype(size) [DEFAULT default_value];
```

ตัวอย่าง

```
CREATE PROCEDURE `GetTotalAccount`()
BEGIN
    DECLARE totalAccount INT DEFAULT 0;

    SELECT COUNT(*)
    INTO totalAccount
    FROM account;

    SELECT totalAccount;
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`GetTotalAccount`();
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

totalAccount
2

3. MySQL Stored Procedure Parameters

- คำสั่งสำหรับการจัดการพารามิเตอร์

```
[IN | OUT | INOUT] parameter_name datatype[(length)]
```

<u>**ตัวอย่าง**</u>

```
CREATE PROCEDURE `GetBranchAsset`(
    IN  b_name VARCHAR(20),
    OUT b_asset float
)
BEGIN
    SELECT asset
    INTO b_asset
    FROM branch
    WHERE branch_name = b_name;
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`GetBranchAsset`('SUT', @b_asset);
SELECT @b_asset;
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

@b_asset
1000000

4. MySQL IF Statement

- คำสั่ง simple IF-THEN statement

```
IF condition THEN
   statements;
END IF;
```

ตัวอย่าง

```
CREATE PROCEDURE `GetAccountLevel`(
    IN  a_number INT,
    OUT a_level  VARCHAR(20))
BEGIN
    DECLARE a_balance FLOAT DEFAULT 0;

    SELECT balance
    INTO a_balance
    FROM account
    WHERE account_number = a_number;

    IF a_balance > 150 THEN
        SET a_level = 'PLATINUM';
    END IF;
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`GetAccountLevel`(3, @a_level);
SELECT @a_level;
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

@a_level
PLATINUM

5. MySQL Loops
   - คำสั่งการสร้าง LOOP

```
[label]: LOOP
    ...
    -- terminate the loop
    IF condition THEN
        LEAVE [label];
    END IF;
    ...
END LOOP;
```

- คำสั่งการสร้าง WHILE Loop

```
[begin_label:] WHILE search_condition DO
    statement_list
END WHILE [end_label]
```

- คำสั่งการสร้าง REPEAT Loop

```
[begin_label:] REPEAT
    statement
UNTIL search_condition
END REPEAT [end_label]
```

<u>ตัวอย่าง</u>

```
CREATE PROCEDURE `LoopDemo`()
BEGIN
    DECLARE x  INT;
    DECLARE str  VARCHAR(255);

    SET x = 1;
    SET str =  '';

    loop_label:  LOOP
        IF  x > 10 THEN
            LEAVE  loop_label;
        END  IF;

        SET  x = x + 1;
        IF  (x mod 2) THEN
            ITERATE  loop_label;
        ELSE
            SET  str = CONCAT(str,x,',');
        END  IF;
    END LOOP;
    SELECT str;
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`LoopDemo`();
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

str
2,4,6,8,10,

6. MySQL Error Handling in Stored Procedures

- คำสั่งสำหรับจัดการ Error

```
DECLARE action HANDLER FOR condition_value statement;
```

- Action
  - CONTINUE : the execution of the enclosing code block (BEGIN ... END) continues.
  - EXIT : the execution of the enclosing code block, where the handler is declared, terminates.
- condition_value
  - SQLSTATE
  - SQLWARNING
  - NOTFOUND
  - SQLEXCEPTION

<u>ตัวอย่าง</u>

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `InsertBranch`(
    IN  b_name VARCHAR(20),
    IN  b_city VARCHAR(20),
    IN  b_asset FLOAT
)
BEGIN
    DECLARE EXIT HANDLER FOR 1062
    BEGIN
        SELECT CONCAT('Duplicate key (',b_name,') occurred') AS message;
    END;

    INSERT INTO branch
    VALUES(b_name, b_city, b_asset);
END
```

เรียกใช้งาน stored procedure

```
CALL `db_lab7`.`InsertBranch`('SUT', 'Korat', 30000000);
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

message
Duplicate key (SUT) occurred

7. MySQL Stored Function

- คำสั่งสร้าง stored function

```
CREATE FUNCTION function_name(
    param1,
    param2,…
)
RETURNS datatype
[NOT] DETERMINISTIC
BEGIN
 -- statements
END
```

- คำสั่งลบ stored function

```
DROP FUNCTION [IF EXISTS] stored_function_name;
```

- คำสั่งดู stored function ในระบบ

```
SHOW FUNCTION STATUS [LIKE 'pattern' | WHERE search_condition]
```

<u>ตัวอย่าง</u>

```
CREATE FUNCTION `AccountLevel`(
    a_balance FLOAT
) RETURNS varchar(20) CHARSET utf8
    DETERMINISTIC
BEGIN
    DECLARE accountLevel VARCHAR(20);

    IF a_balance > 150 THEN
        SET accountLevel = 'PLATINUM';
    ELSEIF (a_balance >= 100 AND
            a_balance <= 150) THEN
        SET accountLevel = 'GOLD';
    ELSEIF a_balance < 100 THEN
        SET accountLevel = 'SILVER';
    END IF;

    RETURN (accountLevel);
END
```

เรียกใช้งาน stored function

```
SELECT
    account_number,
    balance,
    AccountLevel(balance)
FROM
    account
ORDER BY
    account_number;
```

ผลลัพธ์

Result Grid Filter Rows: Search Export:

account_number	balance	AccountLevel(balan...
1	150	GOLD
3	200	PLATINUM